



Society of Petroleum Engineers

SPE-227268-MS

Leveraging GPU-Based High-Performance Computing for Enhanced Wavefield Traveltime Inversion Algorithms

Young Seo Kim, Maxim Dmitriev, and Hussain Salim, EXPEC Advanced Research Center, Saudi Aramco, Dhahran

Copyright 2025, Society of Petroleum Engineers DOI [10.2118/227268-MS](https://doi.org/10.2118/227268-MS)

This paper was prepared for presentation at the Middle East Oil, Gas and Geosciences Show (MEOS GEO), Manama, Bahrain, 16 – 18 September 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Abstract

Full Waveform Inversion (FWI) has become a widely accepted method for accurate velocity estimation in seismic imaging and reservoir modeling, but it still faces challenges, particularly in building a suitable starting model and managing computational resources. To overcome these limitations, we have developed the Wavefield Traveltime Inversion (WTI) algorithm, which provides a reliable initial model for FWI. We have optimized the WTI algorithm for efficiency on a Graphics Processing Unit (GPU) system by rewriting its functions in the CUDA programming language, employing a GPU-based lossy compression library to reduce memory requirements, and utilizing multiple CUDA streams for enhanced performance. Additionally, we have modified the task distribution system to better suit the WTI program, ensuring that computational resources are utilized efficiently. The resulting GPU-based WTI algorithm has been applied to a 3D land dataset, demonstrating its capabilities in providing accurate and detailed velocity models, and highlighting its potential as a valuable tool for seismic imaging and reservoir modeling, particularly when used in conjunction with FWI.

Introduction

High-resolution velocity models are essential for minimizing risks associated with geological interpretations and drilling hazards. Full Waveform Inversion (FWI) is a powerful method for constructing these models, as introduced by [Tarantola \(1984\)](#). However, FWI has significant limitations, including a substantial computational burden due to repetitive iterations and the need for a wave equation solver. Additionally, FWI requires an accurate starting model, which can be challenging to obtain when the field data lacks low-frequency components below 3 Hz. To address these limitations, we have developed the Wavefield Traveltime Inversion (WTI) algorithm, building on the work of [Luo and Schuster \(1991\)](#). The WTI algorithm is designed to provide a reliable initial model for FWI, leveraging the available frequency information from 3D field datasets, which typically ranges from 6 Hz onwards. To enhance the stability, robustness, and performance speed of the WTI algorithm, we have integrated several key enhancements. These enhancements include the implementation of an Eikonal solver, which provides a stable and efficient method for picking first-arrival times. We have also developed CUDA-programmed libraries to harness the capabilities of Graphics Processing Units (GPUs), enabling significant acceleration of the computation.

Furthermore, we have incorporated lossy data compression techniques to improve data input/output (I/O) within the WTI algorithm, reducing the computational burden and enhancing overall performance. By addressing the limitations of FWI and providing a reliable initial model, the WTI algorithm offers a powerful tool for constructing high-resolution velocity models. The integration of these enhancements has significantly improved the stability, robustness, and performance speed of the WTI algorithm, making it an attractive solution for a wide range of geological and geophysical applications.

Brief algorithm of wavefield traveltime inversion (WTI)

The objective function utilized in Waveform Tomography Inversion (WTI) is constructed by calculating the temporal difference between the first arrivals of synthetic and field data. This is achieved through the following equation, as initially proposed by [Luo and Schuster \(1991\)](#):

$$E_{WTI}(\mathbf{m}) = \frac{1}{2} \sum_{j=1}^{n_r} \|t_{u,j}(\mathbf{m}) - t_{d,j}\|^2 = \frac{1}{2} \sum_{j=1}^{n_r} \|\Delta t_j\|^2 \quad (1)$$

where n_r is the number of receivers, $\mathbf{m}$ is velocity and $t_{u,j}$ and $t_{d,j}$ are first-arrival times picked from modeled and field data at the j receiver, respectively, and Δt_j is the time difference between two first-arrival times.

There are several approaches to obtain the time difference Δt_j . [Chi et al. \(2015\)](#) and [Luo et al. \(2016\)](#) have employed cross-correlation of modeled and field traces to calculate Δt_j . [Ma and Hale \(2013\)](#) have utilized dynamic image warping, as introduced by [Hale \(2013\)](#), to construct the time difference. However, these automatic methods can be susceptible to errors, particularly in areas with complex geological structures in the shallow subsurface and strong seismic noises adjacent to the first arrivals. To mitigate these risks, we have adopted a hybrid approach, combining an automatic picker with additional manual quality control (QC) checks to pick the first-arrival times from the data. For the synthetic first-arrival travel times, we employ an in-house automatic picker that utilizes the Modified Energy Ratio (MER) algorithm, as described by [Wong et al. \(2009\)](#), in conjunction with a 2D tracking algorithm.

The adjoint source $g_{WTI}(t)$ can be expressed as follows:

$$g_{WTI}(t) = \Delta t \frac{\frac{\partial u(t)}{\partial t}}{\int \frac{\partial u(t)}{\partial t} \cdot \frac{\partial u(t)}{\partial t} dt} \quad (2)$$

where $u(t)$ is the synthetic modeled data. Actually, $\frac{\partial u(t)}{\partial t}$ can be replaced with $\frac{\partial d(t + \Delta t)}{\partial t}$ where $d(t)$ is the field data. However, in this study, we have chosen to use [Equation 2](#) as the adjoint source, which avoids the need for source estimation, as discussed by [Kim et al. \(2013\)](#), by utilizing the field data during the inversion process. The quality of the WTI results is heavily dependent on the accuracy of the first break picks, which can be a challenging task, especially in complex geological settings. To address this issue, iterative re-picking of the first breaks with modeled first arrivals as a guidance, followed by re-running the WTI with the newly revised picks, is often necessary to improve the accuracy of the final WTI model.

The WTI gradient vector, which is essential for updating the velocity model during the inversion process, can be expressed as follows:

$$\frac{\partial E_{WTI}}{\partial \mathbf{m}} = \int_0^{T_{\max}} \partial^2 u / \partial t^2 \cdot \hat{u}_g dt \quad (3)$$

where $\hat{u}_g$ is the receiver wavefield back-propagated by the adjoint source $g_{WTI}(t)$ and u is the forward wavefield obtained by propagating a Ricker wavelet as a source. This equation provides the gradient of the

objective function with respect to the model parameters, which is used to update the velocity model in an iterative manner, ultimately leading to a more accurate representation of the subsurface.

Proposed WTI algorithm with Eikonal solver

The computation of traveltimes is a crucial element in the WTI objective function calculation. While traveltimes can be derived using a picker program, its application for generating synthetic first arrival traveltimes may introduce instability, particularly when dealing with complex geological structures. Therefore, opting for a traveltimes algorithm grounded in seismic ray theory is a prudent choice to ensure stability during the iterative WTI process. One notable approach for traveltimes computation is the Fast Marching Method (FMM), which leverages first-order upwind finite differences within the context of isotropic front propagation, as introduced by [Sethian and Popovici \(1999\)](#). However, the FMM's reliance on irregular data updates and a heterogeneous data structure poses challenges for parallel implementations, especially on GPU architectures. Such GPUs excel when numerous computing cores can execute identical operations concurrently on distinct data. Notably, the FMM's demands, such as the need to update grid points sequentially and the inability to parallelize heap updates, impede its efficient parallel execution.

A contemporary alternative, the Fast Sweeping Method (FSM), presents an iterative algorithm employing Gauss-Seidel iterations with alternating sweeping orders. It addresses a discretized Eikonal equation on a rectangular grid, as introduced by [Zhao \(2004\)](#). Interestingly, when both methods employ the same first-order upwind discretization, the computed solutions from the Fast Sweeping Method are indistinguishable from those generated by the Fast Marching Method. [Detrixhe et al. \(2013\)](#) propose a sweeping strategy that introduces a novel parallel approach. This strategy groups diverse grid points in a manner conducive to simultaneous updates of point sets. Their parallel technique incorporates a Cuthill-McKee ordering scheme for distributing grid points among available processes, facilitating a more convenient parallelization of computation tasks for developers.

In pursuit of enhanced traveltimes computation efficiency, we have developed a 3D GPU-based Eikonal solver using CUDA toolkits, building upon insights from Detrixhe's work. A Godunov upwind differencing scheme on the interior nodes can be expressed as:

$$\left[\left(u_{i,j}^{new} - u_{xmin} \right)^+ \right]^2 + \left[\left(u_{i,j}^{new} - u_{ymin} \right)^+ \right]^2 = f_{i,j}^2 h^2, \quad i = 2, \dots, I-1; j = 2, \dots, J-1, \quad (4)$$

where h is the grid spacing and the following notations are used:

$$u_{xmin} = \min(u_{i-1,j}, u_{i+1,j}), \text{ and } (x)^+ = \begin{cases} x, & x > 0, \\ 0, & x \leq 0. \end{cases} \quad (5)$$

The algorithm proceeds by sweeping through all the nodes and assigning a new value to u by solving the equation and updating the solution as $u_{i,j} = \min(u_{i,j}, u_{i,j}^{new})$.

The table below presents a performance comparison between FMM and FSM, with runtime tests conducted using a 3D constant gradient velocity model. As shown in [Table 1](#), the results confirm that the acceleration ratio increases as the grid size increases.

Table 1—The performance comparison between Fast March method and Fast Sweep method depending on grid sizes. Algorithms for FMM and FSM are written with CPU-based C++ and NVIDIA GPU-based CUDA languages, respectively.

Grid Size	101x101x101	201x201x201	501x501x501
Fast March(CPU)	0.472s	5.111s	132.349s
Fast Sweep(GPU)	0.264s	0.772s	16.205s
Acceleration Ratio	1.79	6.62	8.17

In an effort to further refine the existing WTI algorithm, our research endeavors have focused on enhancing its compatibility with contemporary CPU/GPU heterogeneous architectures. A crucial aspect of this endeavor has been the incorporation of the Fast Sweep Method (FSM) Eikonal solver for synthetic traveltimes calculation, leveraging the strengths of both techniques to achieve more accurate and efficient results. To this end, we have successfully harnessed the power of GPU lossy compression through Nvidia's proprietary Bitcomp compressor, enabling the efficient utilization of D-RAM or V-RAM for snapshot storage. The WTI algorithm, which parallels Full Waveform Inversion (FWI) and Reverse Time Migration (RTM), engages in both forward and backward wave equation-based propagation. During the forward propagation phase, the forward (source) wavefield necessitates storage in either hard disk or computer memory, to be subsequently loaded during backward propagation for gradient vector computation, essential for velocity updates. However, I/O operations pertaining to forward wavefields can significantly impede WTI's performance, as the repeated loading and storing of large datasets can lead to substantial bottlenecks. To mitigate this issue, the integration of the BitComp compressor has proven instrumental in reducing the stored forward wavefield's size, thereby improving overall performance. The compression technique efficiently manages the storage of snapshots in D-RAM or V-RAM, allowing for faster access and manipulation of the data. This, in turn, enables the WTI algorithm to operate more efficiently, particularly during the forward propagation phase, where the forward wavefield is stored and later loaded for gradient vector computation. By minimizing the storage requirements and optimizing data access, the BitComp compressor plays a vital role in accelerating the WTI process, making it more suitable for large-scale seismic data processing.

In addition to the compression technique, we have devised an improved workflow tailored specifically for WTI, which is particularly advantageous when employing GPUs. Our WTI implementation gains additional acceleration by utilizing multiple streams under GPUs, allowing for concurrent execution of various tasks and maximizing the utilization of GPU resources. Within a time loop, three streams are introduced: the first for the Finite Difference Method (FDM) kernel, responsible for simulating the wave propagation; the second for extracting the forward wavefield at receiver locations to calculate the objective function and its adjoint source; and the third for compression/uncompression of the second-derivative of the forward wavefield and gradient vector calculation. This multi-stream approach enables the efficient execution of multiple tasks simultaneously, reducing the overall computational time and improving the performance of the WTI algorithm. Furthermore, performance enhancements are realized by implementing the finite-difference-based modeling kernel and cPML boundary condition within shared memory, thereby facilitating spatial derivative calculations on neighboring grid points within the two fastest axes. This optimization technique allows for faster and more accurate calculations, contributing to the overall efficiency of the WTI process.

Moreover, we have optimized the equation for the gradient vector by utilizing the first derivative of the wavefield instead of second derivatives, enabling efficient utilization of multiple CUDA streams. This modification not only reduces the computational complexity but also allows for better parallelization of the gradient vector calculation, leading to significant performance improvements. The optimized equation is

designed to take advantage of the GPU's massively parallel architecture, ensuring that the computational resources are utilized efficiently and effectively.

The modified algorithm underpinning our upgraded WTI methodology is visually represented in [Figure 1](#), which illustrates the various components and their interactions. The diagram provides a clear overview of the workflow, highlighting the key steps involved in the WTI process, including forward propagation, backward propagation, and gradient vector computation. By examining the figure, it becomes apparent how the various optimizations and techniques, such as the BitComp compressor and multi-stream approach, contribute to the overall efficiency and performance of the WTI algorithm.

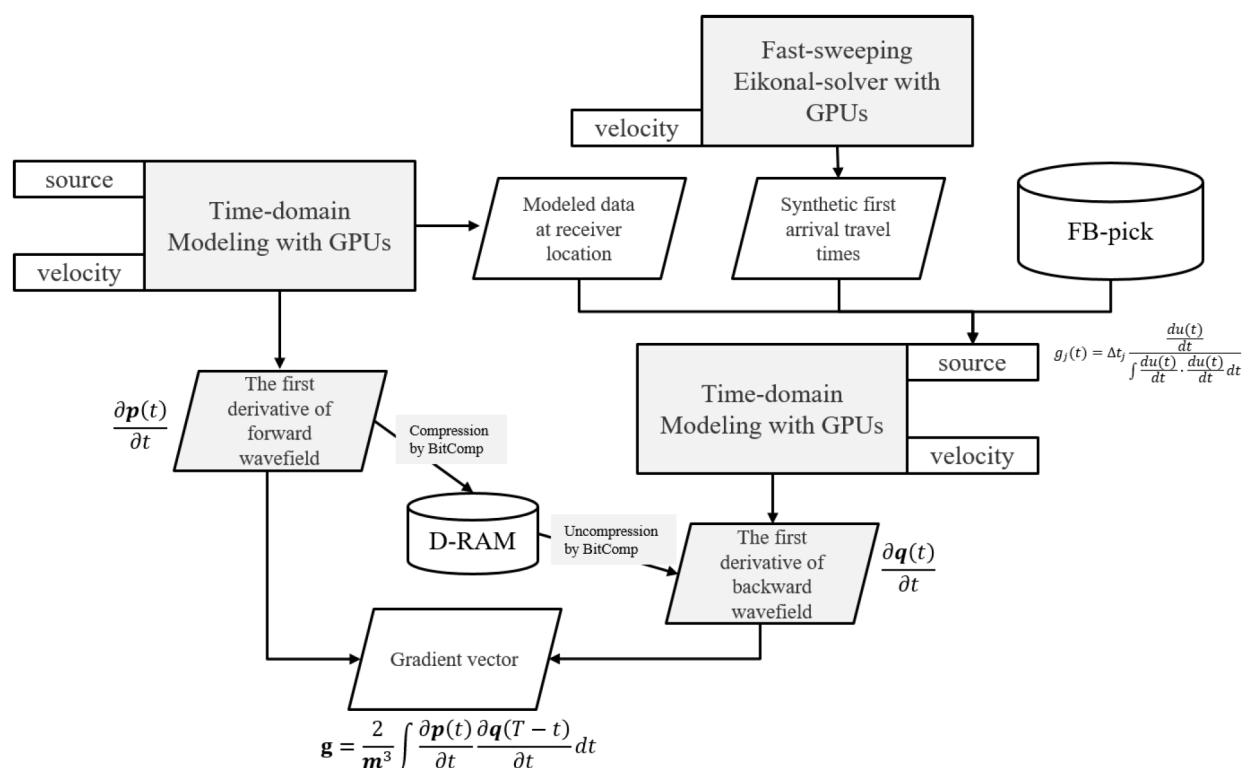


Figure 1—A flowchart illustrating GPU-based WTI algorithm. Gray-colored diagrams are newly revised parts for the performance speedup and improved stability.

Static task scheduling for WTI

The WTI program utilized in this study was initially designed to operate under a dynamic task schedule, which employs a master-slave architecture with multiple CPU cores. In this configuration, the master CPU cores are responsible for sending shot and velocity information to idle slave CPU cores, as well as updating the velocity models. The slave CPU cores, on the other hand, are tasked with executing forward and backward modeling on Graphics Processing Units (GPUs) to obtain a gradient vector for each shot. Following the computation, the slave CPU cores receive the gradient from the GPU device memory and transmit it to the master CPU cores for stacking. However, when applying this dynamic task distribution to WTI for shallow depth model building with small offset ranges, the scalability of multi-GPU systems is limited. This is because the performance time is dominated by the communication overhead associated with sending data and gradients among the master and slave CPU cores. To illustrate this issue, [Figure 2](#) presents the elapsed times for one iteration of waveform inversion using GPUs. Each node in the system is equipped with 8 GPUs (Nvidia Tesla V100 SXM2), and all nodes are interconnected via Nvidia InfiniBand HDR 100. The results show that when WTI is executed with dynamic task scheduling, there is no significant difference in runtime performance between 8 and 24 nodes. This suggests that the dynamic task distribution is not

effectively utilizing the available computational resources, leading to a bottleneck in the system. To address this limitation, we introduced a static task distribution approach. In this configuration, there is no distinction between master and slave cores, and all cores are responsible for generating gradients and accessing data files in parallel. By adopting this static task distribution, the performance speed of the WTI program can be improved as the number of GPUs employed increases. Figure 3 provides a comparison of GPU usage between dynamic and static task distributions when ten nodes are utilized. The vertical axis in each graph represents the percentage of GPU usage, with 100% indicating that all 8 GPUs are fully utilized. The red arrows in the figures denote the starting times of the iteration process. Prior to these times, the program does not utilize the GPUs, as it is engaged in generating a database for the waveform inversion while reading shot gathers and input velocity models.

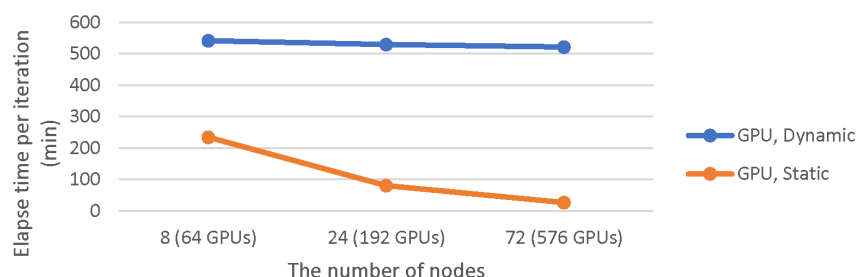


Figure 2—Comparison of elapsed time per iteration between dynamic and static task distribution according to the number of GPUs employed.

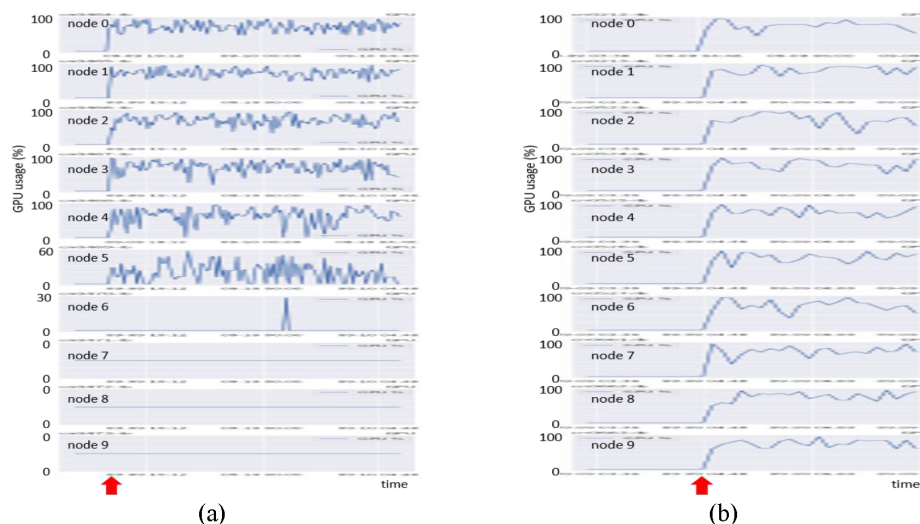


Figure 3—Comparison of GPU usages (%) between (a) dynamic and (b) static task distribution in case 10 nodes (80 GPUs) are employed. Red arrows indicates starting times of the calculation.

A comparison of Figures 3a and 3b reveals that the static task distribution approach yields significantly better GPU usage. In Figure 3b, all ten nodes exhibit around 80-90% GPU usage, indicating that the computational resources are being utilized with satisfaction. In contrast, when using the dynamic task distribution, nodes 6 to 9 are idle, and the maximum GPU usage on node 5 reaches only 60%. This result provides insight into why there is no runtime difference between 8, 24, and 72 nodes when using dynamic task distribution, as shown in Figure 2. The static task distribution approach effectively overcomes the communication bottleneck and enables the WTI program to scale more efficiently with increasing numbers of GPUs. The benefits of the static task distribution approach can be attributed to the reduced communication overhead and improved parallelization of the computation. By eliminating the distinction between master and slave cores, the static task distribution enables all cores to contribute to the computation, resulting in

a more efficient utilization of the available resources. Furthermore, the parallel reading of data files and generation of gradients enable the program to take full advantage of the multi-GPU architecture, leading to significant performance improvements. Overall, the introduction of the static task distribution approach has enabled the WTI program to achieve better scalability and performance, making it more suitable for large-scale seismic data processing applications.

Numerical Example on 3D land dataset

We applied the WTI algorithm to a land data set with the objective of building a depth velocity model. The WTI workflow utilized only the first-arrival traveltimes from the field data, which limited the penetration depth to approximately $\frac{1}{4}$ of the maximum offset. However, it is essential to note that an accurate shallow model is crucial for building the deeper section of the model using reflection tomography with geological interpretation or Full Waveform Inversion (FWI) with reflection events. To initiate the WTI process, we obtained first break picks using an automatic picker, which were then subjected to quality control (QC) checks to ensure their accuracy. The picked times were then used as input for the WTI algorithm, along with a geo-statistical velocity model as the initial model. Figure 4a shows a depth slice extracted from the geo-statistical model at 100 m below the average topography elevation, providing a starting point for the inversion process.

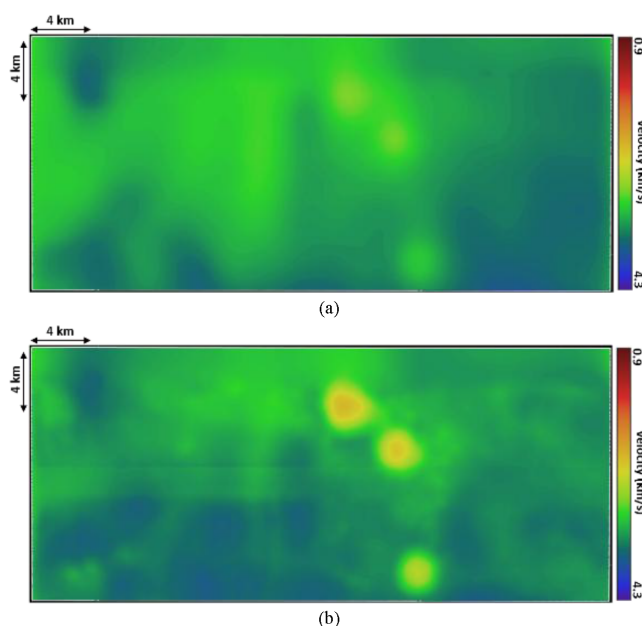


Figure 4—Depth slices at 100 m depth extracted from (a) the geostatistical model and (b) the WTI model

In the WTI implementation, the grid sizes were set to 40 m, 40 m, and 20 m in the inline, xline, and vertical directions, respectively. A Ricker wavelet with a dominant frequency of 6 Hz was used as the source wavelet, and the minimum and maximum offsets were set to 0 m and 2,200 m, respectively.

Figure 4b shows a depth slice extracted from the WTI velocity model at the 12th iteration. Compared to the initial model, three distinct circle-shaped velocity collapses are clearly visible in Figure 4b. These features are likely indicative of subsurface geological structures, such as faults or fractures, that were not well-resolved in the initial model. Additionally, two horizontal stripes are visible on the depth slice, which are attributed to the boundary between two surveys with different acquisition parameters. Although a balancing operator was applied to the gradient vector during the inversion process to mitigate the effects of survey boundaries, the stripes are still visible due to the inherent differences in the acquisition parameters. The presence of these stripes highlights the challenges associated with integrating data from multiple

surveys, particularly when the acquisition parameters differ significantly. However, the WTI algorithm has successfully imaged the subsurface velocity structure, including the circle-shaped velocity collapses, which are essential for building an accurate depth velocity model. The results demonstrate the effectiveness of the WTI algorithm in capturing the subsurface geological features, even in the presence of survey boundaries and limited penetration depth.

To ensure the quality and accuracy of the first break traveltime picks, we conducted a comparison between the field data and synthetic modeled data. This QC step is essential in verifying the reliability of the picks and identifying any potential issues or discrepancies. Figure 5 presents the results of this comparison, displaying the picked first breaks on the field data and synthetic first arrival traveltimes overlaid with a synthetic shot gather sorted by signed offset. The figure shows two separate comparisons: one between the field data and the geo-statistical model (Figure 5a), and another between the field data and the WTI model (Figure 5b). In Figure 5a, we observe that the first arrivals generated from the geo-statistical model match quite well with the picked first arrivals from the field data for the short offset range. This suggests that the geo-statistical model is able to accurately capture the near-surface velocity structure, which is essential for building a reliable depth velocity model. However, as we move to larger offset ranges (greater than 1 km), we notice a significant difference between the two travel times. This discrepancy indicates that the geo-statistical model may not be accurately representing the deeper subsurface structure, which can lead to errors in the subsequent inversion processes. In contrast, Figure 5b shows that the first arrival times from the WTI model are in excellent agreement with the picked first-arrival times along all offset ranges. This suggests that the WTI model has successfully captured the subsurface velocity structure, including both the near-surface and deeper sections. The good agreement between the WTI model and the field data indicates that the WTI algorithm has been able to accurately invert the first break traveltime picks, resulting in a reliable and accurate depth velocity model.

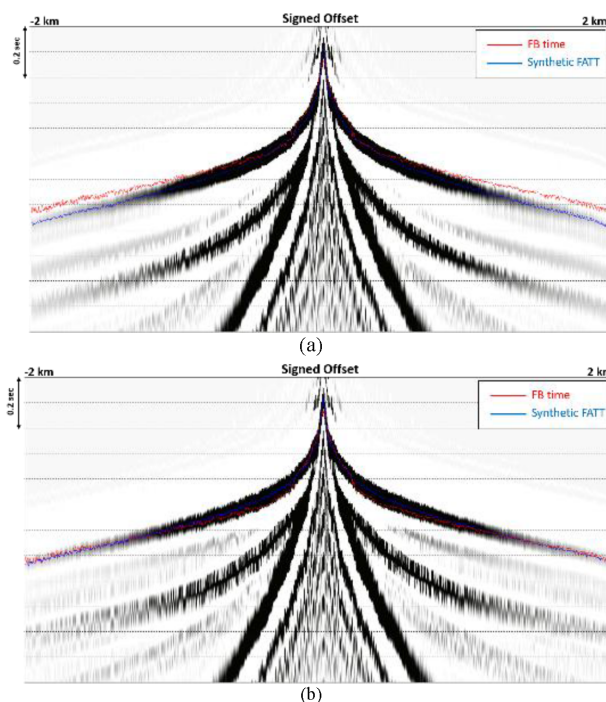


Figure 5—Comparison of first break picks (FBs) and synthetic first arrival travel times (synthetic FATTs) generated from (a) the initial model and (b) the final WTI model. The red and blue lines overlaid on synthetic traces indicate FBs and synthetic FATTs, respectively.

Conclusion

We have developed a 3D WTI program optimized for enhanced performance on Graphics Processing Units (GPUs). Our approach involves implementing a finite-difference-based modeling kernel and a convolutional Perfectly Matched Layer (cPML) boundary condition, which enables efficient and accurate modeling of seismic wave propagation. To accelerate computation, we utilize shared memory to calculate spatial derivatives on neighboring grid points, reducing memory access latency and improving performance. We have also incorporated a GPU lossy compression technique called Bitcomp, which enables efficient utilization of memory for snapshot storage, reducing the memory footprint and improving performance. Additionally, we use multiple CUDA streams to accelerate computation by overlapping memory transfers and computations, reducing overall computation time. Our GPU-based WTI algorithm is designed to take advantage of the massively parallel architecture of modern GPUs, enabling fast and efficient computation of large-scale seismic datasets. We applied our algorithm to a 3D onshore dataset obtained in a challenging geological setting, where the dataset presents several challenges, including complex geology and limited seismic data quality. To validate the improvement of the WTI inverted model, we compared the first arrival travel times of synthetic and field traces, and the results show that the WTI inverted model accurately predicts the first arrival travel times, indicating a significant improvement in the accuracy of the velocity model. By using GPUs and static task distribution, we achieved significant performance improvements, accelerating the computation of the WTI algorithm and reducing the overall computation time. The static task distribution approach enables us to divide the computation into smaller tasks, which can be executed concurrently on multiple GPUs, further improving performance and reducing computation time.

References

- Chi, B., L. Dong and Y. Liu. 2015, Correlation-based reflection full-waveform inversion, *Geophysics*, **80**(4), R189–R202.
- Detrixhe, M., F. Gibou, C. Min. 2013, A parallel fast sweeping method for the Eikonal equation, *J. Comput. Phys.*, vol. **237**, pp. 46–55.
- Kim, Y., Y. Cho, and C. Shin, 2013, Estimated source wavelet-incorporated reverse-time migration with a virtual source imaging condition, *Geophysical Prospecting*, **61**, 317–333.
- Luo, Y. and G.T. Schuster, 1991, Wave-equation travelttime inversion, *Geophysics*, **56**(5), 645–653.
- Sethian, J. and A. Popovici, 1999, 3-D travelttime computation using the fast marching method, *Geophysics*, **65**, 516–523.
- Tarantola, A., 1984, Inversion of seismic reflection data in the acoustic approximation, *Geophysics*, **49**, 1259–1266.
- Wong, J., L. Han, J. Bancroft, and R. Stewart. 2000, Automatic time-picking of first arrivals on large seismic datasets, *CREWES, Univ. Calgary, Calgary, AB, Canada, Tech. Rep.*, **26**, 14.
- Zhao, H. 2004, A fast sweeping method for eikonal equations, *Math. Of Comput.*, vol. **74**, n. 250, pp. 603–627.